

Reactive Database Access with R2DBC

Why JDBC Kills a Reactive App

JDBC was designed in 1997 for synchronous, blocking I/O. It has no concept of a reactive publisher. R2DBC was designed from scratch in 2018 specifically for reactive runtimes. You cannot use JDBC drivers with WebFlux without offloading to a separate thread pool, which defeats the purpose.

```
// This looks reactive but is broken
@GetMapping("/products/{id}")
public Mono<Product> getProduct(@PathVariable Long id) {
    return Mono.fromCallable(() -> jpaRepository.findById(id).orElseThrow());
    // jpaRepository.findById() calls JDBC under the hood
    // JDBC blocks the thread
    // You just blocked a Netty event loop thread
    // That thread can't handle any other requests while blocked
    // You've defeated the entire purpose of WebFlux
}
```



Blocking a Netty event loop thread is significantly worse than blocking a Tomcat thread. Tomcat has 200 threads and losing one to a block costs 0.5% capacity. Netty has 16 threads on an 8-core machine meaning losing one to a block costs 6.25% capacity. One rogue JDBC call can measurably degrade your entire server.

Dependencies

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>r2dbc-postgresql</artifactId>
  <scope>runtime</scope>
</dependency>

```



If your application does not need streaming or backpressure, skip R2DBC entirely. Standard JDBC with virtual threads enabled gives you the exact same `User`, `@Entity`, and `JpaRepository` you already know, full relationship support, and `ddl-auto` for schema generation. You lose nothing except the ability to stream massive result sets.

Configuration

```

spring:
  r2dbc:
    url: r2dbc:postgresql://localhost:5432/users_db
    username: postgres
    password: postgres
    pool:
      initial-size: 5
      max-size: 20
      max-idle-time: 30m
  sql:
    init:
      mode: always
      schema-locations: classpath:schema.sql

```

Schema Generation

```
CREATE TABLE IF NOT EXISTS users (
    id          BIGSERIAL PRIMARY KEY,
    name        VARCHAR(255) NOT NULL,
    email       VARCHAR(255) NOT NULL UNIQUE,
    role        VARCHAR(100) NOT NULL DEFAULT 'USER',
    active      BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
```

The connection pool is critical in production. R2DBC uses a reactive connection pool under the hood. Without configuring pool size, you will hit connection exhaustion under load.

The Domain Entity

R2DBC entities look similar to JPA entities but with important differences. There is no `@Entity`, no lazy loading, no `@OneToMany`. R2DBC is intentionally simple and does not support JPA-style relationships.

```
@Table("users")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class User {

    @Id
    private Long id;

    private String name;
    private String email;
    private String role;
    private boolean active;

    @CreatedDate
```

```

@Column("created_at")
private LocalDateTime createdAt;

@LastModifiedDate
@Column("updated_at")
private LocalDateTime updatedAt;
}

```

Key differences from JPA:

- `@Id` is from `org.springframework.data.annotation`, not `jakarta.persistence`
- `@CreatedDate` and `@LastModifiedDate` require `@EnableR2dbcAuditing` on a config class
- No `@GeneratedValue` needed, R2DBC detects `null` id and treats it as insert

⚡ None of these differences exist with JDBC and virtual threads. You keep `@Entity`, `@OneToMany`, `@ManyToOne`, lazy loading, and the JPA dirty checking mechanism exactly as before. The only reason to give up all of this is if you specifically need reactive streaming.

Enabling Auditing

```

@Configuration
@EnableR2dbcAuditing
public class R2dbcConfig {
}

```

Without this, `@CreatedDate` and `@LastModifiedDate` will never be populated.

⚡ R2DBC has no dirty checking like JPA. Every `save()` call issues an UPDATE for the entire row regardless of what changed. There is no session, no persistence context, no first-level cache. Every operation is a fresh database round trip.

The Repository

```
public interface UserRepository extends ReactiveCrudRepository<User, Long>,
                                ReactiveSortingRepository<User, Long> {

    Flux<User> findByRole(String role);

    Flux<User> findByActive(boolean active);

    Mono<User> findByEmail(String email);

    Mono<Boolean> existsByEmail(String email);

    @Query("SELECT * FROM users WHERE name ILIKE :name ORDER BY created_at")
    Flux<User> searchByName(String name);

    @Query("SELECT COUNT(*) FROM users WHERE role = :role")
    Mono<Long> countByRole(String role);
}
```

- `ReactiveCrudRepository` gives you `findById`, `findAll`, `save`, `delete`, `count`
- `PagingAndSortingRepository` gives you `findAll(Sort)`
- Derived query methods work the same as Spring Data JPA
- `@Query` for anything that cannot be expressed as a derived method

The Service Layer

Continuing with the same `UserService` from the previous sections:

```
@Slf4j
@Service
@RequiredArgsConstructor
```

```

public class UserService {

    private final UserRepository userRepository;

    public Flux<UserResponse> findAll() {
        return userRepository.findAll()
            .map(UserResponse::from)
            .doOnSubscribe(s -> log.info("Fetching all users"));
    }

    public Mono<UserResponse> findById(Long id) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(new UserNotFoundException(id)))
            .map(UserResponse::from);
    }

    public Mono<UserResponse> create(UserRequest request) {
        return userRepository.existsByEmail(request.getEmail())
            .flatMap(exists -> {
                if (exists) return Mono.error(
                    new DuplicateEmailException(request.getEmail()));

                User user = User.builder()
                    .name(request.getName())
                    .email(request.getEmail())
                    .role(request.getRole())
                    .active(true)
                    .build();
                return userRepository.save(user);
            })
            .map(UserResponse::from)
            .doOnSuccess(u -> log.info("Created user id: {} ", u.getId()));
    }
}

```

```

    }

    public Mono<UserResponse> update(Long id, UserRequest request) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(new UserNotFoundException(id)))
            .flatMap(existing -> {
                existing.setName(request.getName());
                existing.setEmail(request.getEmail());
                existing.setRole(request.getRole());
                return userRepository.save(existing);
            })
            .map(UserResponse::from);
    }

    public Mono<Void> delete(Long id) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(new UserNotFoundException(id)))
            .flatMap(userRepository::delete);
    }

    public Flux<UserResponse> findByRole(String role) {
        return userRepository.findByRole(role)
            .map(UserResponse::from);
    }

    public Flux<UserResponse> search(String name) {
        return userRepository.searchByName("%" + name + "%")
            .map(UserResponse::from);
    }
}

```

Notice the duplicate email check uses `existsByEmail` returning `Mono<Boolean>` and then `flatMap` to branch. This is a very common reactive pattern for conditional

logic before a write.

Transactions

R2DBC supports reactive transactions via `@Transactional`. Spring wraps the reactive pipeline in a transaction context automatically.

```
@Transactional
public Mono<UserResponse> createWithProfile(UserRequest request) {
    return userRepository.save(buildUser(request))
        .flatMap(savedUser -> profileRepository
            .save(buildProfile(savedUser.getId(), request)))
        .thenReturn(savedUser)
        .map(UserResponse::from);
}
```

If any step in the chain throws or returns an error signal, the transaction rolls back automatically. The key rule is that you must stay within the reactive pipeline. If you subscribe inside a `@Transactional` method and start a new pipeline, the transaction context will not propagate.

```
// wrong -- inner subscribe breaks transaction boundary
@Transactional
public Mono<UserResponse> create(UserRequest request) {
    userRepository.save(buildUser(request)).subscribe(); // transaction lost here
    return Mono.just(new UserResponse());
}

// correct -- single unbroken pipeline
@Transactional
public Mono<UserResponse> create(UserRequest request) {
    return userRepository.save(buildUser(request))
}
```

```
        .map(UserResponse::from);  
    }
```



`@Transactional` works the same way conceptually in both worlds, but reactive transaction propagation is fragile. Breaking the pipeline with a stray `.subscribe()` silently loses the transaction with no compile-time warning. JDBC transactions with virtual threads use the same proven `@Transactional` semantics that have existed for two decades, no special reactive pipeline discipline required.

Streaming Data from the Database

This is where R2DBC genuinely shines over JDBC. Because `findAll()` returns a `Flux`, rows are emitted as they arrive from the database. The entire result set is never loaded into memory.

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)  
public Flux<UserResponse> streamAllUsers() {  
    return userService.findAll();  
}
```

Under the hood R2DBC uses a cursor with a configurable fetch size. Rows are fetched in batches from PostgreSQL as the downstream subscriber requests them.

Control fetch size per connection:

```
spring:  
  r2dbc:  
    properties:  
      fetchSize: 1000
```

This means for 1 million rows, your JVM heap holds at most 1000 rows at a time. The rest stay in PostgreSQL until requested by backpressure from the subscriber.



This is the one place virtual threads cannot fully replace R2DBC. JDBC with virtual threads still loads the entire result set into a `List`. If you are streaming millions of rows to a client, R2DBC is the better tool.

Handling Joins Manually

R2DBC has no relationship mapping. If you need data from two tables together, you write the SQL and map the result yourself.

```
@Query("""
    SELECT u.id, u.name, u.email, p.bio, p.avatar_url
    FROM users u
    LEFT JOIN profiles p ON p.user_id = u.id
    WHERE u.id = :id
    """)
Mono<UserWithProfileDto> findUserWithProfile(Long id);
```

```
@Data
public class UserWithProfileDto {
    private Long id;
    private String name;
    private String email;
    private String bio;

    @Column("avatar_url")
    private String avatarUrl;
}
```

Spring Data R2DBC maps flat query results to DTOs directly if the field names match column names.

R2DBC vs JPA

	JPA + JDBC	R2DBC
Thread model	Blocking, thread waits on I/O	Non-blocking, thread never waits
Relationships	Full support, <code>@OneToMany</code> etc	Not supported, manual joins only
Dirty checking	Automatic	None, always full update
Schema generation	<code>ddl-auto</code>	Manual SQL only
Query language	JPQL, Criteria API, raw SQL	Raw SQL only
Streaming large results	Loads all into memory	Native backpressure streaming
Maturity	Very mature, decades of tooling	Relatively new, evolving
Learning curve	Moderate	High, requires reactive mindset
Community resources	Vast	Limited

When R2DBC Is the Right Choice

- Streaming large result sets to clients where backpressure matters
- Full WebFlux stack where blocking JDBC would break the reactive pipeline
- Data pipelines where rows are processed as they arrive, not collected into a list

When to Stay on JPA + JDBC

- Standard CRUD with relational data, foreign keys, and joins
- You need `@OneToMany`, lazy loading, or any relationship traversal
- You are on Spring MVC with virtual threads, same concurrency benefits, none of the trade-offs



The most common mistake teams make is adopting R2DBC simply because they are using WebFlux, assuming the two must go together. They do not. You can use WebFlux with a blocking JDBC call offloaded to a bounded scheduler, and many production teams do exactly that. R2DBC earns its place only when you specifically need non-blocking database I/O with backpressure streaming. If you cannot point to that need in your application, JPA with virtual threads is the pragmatic default now.